

Etapa 3 – Persistência com ORM JavaScript

Data da entrega: 30/09/2026

- **Objetivo:** Implementar a persistência dos dados da aplicação utilizando um banco de dados relacional e um **ORM (Object-Relational Mapping) para JavaScript/Node.js**.
- Configuração do banco de dados relacional (SQLite/PostgreSQL/MySQL).
- Configuração e utilização de um ORM JavaScript, como **Sequelize** ou **Prisma**.
- Criação dos principais **Models/Entities** da aplicação.
- Mapeamento entre objetos JavaScript e tabelas do banco de dados.
- Definição dos relacionamentos entre as entidades.
- Implementação das operações de **CRUD** utilizando o ORM.
- Organização da aplicação utilizando as camadas **Repository + Service**.
- Validação dos dados antes da persistência.
- Implementação das regras de negócio
- Integração da camada de **API + Service**

Entrega

- Aplicação funcionando com banco de dados relacional.
- ORM configurado e integrado ao projeto.
- Models/Entities implementados.
- CRUD completo utilizando o ORM.
- Repository e Service implementados e utilizados pela aplicação.
- Relacionamentos entre as principais entidades funcionando.
- Apresentar a **Modelagem dos Dados**, contendo:
 - MER (Modelo Entidade-Relacionamento);
 - Diagrama de Classes.
- Demonstrar a aplicação funcionando durante a apresentação.
- Código-fonte organizado e disponível no repositório do projeto.

ORM recomendado

Para a disciplina de **Desenvolvimento Web 2**, recomenda-se utilizar um dos seguintes ORMs:

ORM	Vantagem principal	Complexidade	Indicação
Sequelize	Conceito tradicional de ORM e SQL mais evidente	Média	★★★★★

ORM	Vantagem principal	Complexidade	Indicação
Prisma	Excelente experiência de desenvolvimento e suporte a TypeScript	Baixa/Média	★★★★★

Recomendação

Para esta etapa, recomenda-se preferencialmente o **Sequelize**, pois permite compreender de forma mais clara a evolução da persistência:

```
SQL
↓
Driver de Banco de Dados
↓
Repository
↓
ORM
↓
Service
```

Etapa 3

Ao final da Etapa 3, eu recomendaria que os alunos tenham uma estrutura organizada em camadas, deixando claro o papel de Controller, Service, Repository, Model/Entity e Database.

Estrutura sugerida

```
projeto/
├── src/
│   ├── config/
│   │   └── database.js
│   ├── models/
│   │   ├── Usuario.js
│   │   ├── Produto.js
│   │   └── Pedido.js
│   ├── repositories/
│   │   ├── UsuarioRepository.js
│   │   ├── ProdutoRepository.js
│   │   └── PedidoRepository.js
│   └── services/
│       ├── UsuarioService.js
│       ├── ProdutoService.js
│       └── PedidoService.js
```

```
├── controllers/
│   ├── UsuarioController.js
│   ├── ProdutoController.js
│   └── PedidoController.js
├── routes/
│   ├── usuarioRoutes.js
│   ├── produtoRoutes.js
│   └── pedidoRoutes.js
├── app.js
├── server.js
├── database/
│   └── database.sqlite
├── tests/
│   ├── usuario.test.js
│   ├── produto.test.js
│   └── pedido.test.js
├── .env
├── .env.example
├── .gitignore
├── package.json
└── README.md
```

Responsabilidade de cada camada



